DOCUMENTACION TECNICA

RESPALDO Y RESTAURACION FISICOS DE LA BASE DE DATOS OLIMPIADAS

Universidad de San Carlos de Guatemala
Facultad de Ingenieria - Escuela de Ciencias y Sistemas
Sistemas de Bases de Datos 2 - Segundo semestre 2026
Persona 1: Juan Jose Rodas Mansilla

1. OBJETIVO Y ALCANCE

La Fase 2 implementa una estrategia de continuidad para PostgreSQL basada en respaldos fisicos. El objetivo es demostrar que una instancia puede reconstruirse despues de perder su directorio de datos y que la informacion restaurada coincide con la informacion cargada originalmente.

El alcance incluye:

- tres tipos de carga: anio, deporte y deportista;
- una carga inicial y tres cargas posteriores por tipo;
- un respaldo completo seguido por tres respaldos incrementales encadenados;
- restauracion del respaldo completo y de cada prefijo de la cadena;
- validacion por conteos, muestras de filas, integridad referencial y fragmentacion;
- medicion separada de reconstruccion de archivos, recuperacion de PostgreSQL y tiempo total.

Este documento es la documentacion tecnica. El manual de usuario contiene los comandos operativos detallados y las capturas de pantalla se entregan como evidencia independiente.

2. METODOLOGIA

2.1 Preparacion

La ejecucion se realiza desde Proyecto Fase 2/. Se copia .env.example a .env, se generan los lotes con scripts/02-carga/generar_cargas.py y se levanta PostgreSQL 17 mediante Docker Compose.

La generacion de cargas fue verificada en este repositorio. Se obtuvieron 12 tramos, cuatro por tipo:

Tipo	Tramo 01	Tramo 02	Tramo 03	Tramo 04

anio Paris 2024 Tokio 2020 Rio 2016 Londres 2012
deporte 100 m femenino, Paris 2024 100 m femenino, Tokio 2020 100 m femenino, Rio 2016
100 m femenino, Londres 2012
deportista Usain Bolt, Atenas 2004 Usain Bolt, Pekin 2008 Usain Bolt, Londres 2012
Usain Bolt, Rio 2016

Cada tramo contiene solo filas nuevas y sus padres necesarios. Esto evita duplicar datos y mantiene el orden de las llaves foraneas.

2.2 Ciclo experimental

Para cada tipo se ejecuta el siguiente ciclo:

1. eliminar el volumen de datos y crear una instancia vacia;
2. cargar el tramo 01;
3. validar las 15 tablas;
4. crear el respaldo completo;
5. cargar cada tramo 02, 03 y 04;
6. validar y medir fragmentacion despues de cada carga;
7. crear el incremental correspondiente;
8. eliminar nuevamente la instancia;
9. restaurar el respaldo completo o el prefijo full + inc1..incN;
10. validar que los conteos coincidan con la carga original.

El orden carga -> validacion -> respaldo es obligatorio. Un incremental se calcula a partir del manifiesto del respaldo anterior, por lo que no se puede saltar un eslabon de la cadena.

2.3 Evidencia y mediciones

Los scripts imprimen fecha y hora y escriben resultados en logs/. El archivo logs/tiempos.csv conserva las duraciones de cargas, respaldos y restauraciones. La validacion consulta las 15 tablas; la fragmentacion usa pgstattuple y pgstatindex.

3. MODELO ENTIDAD-RELACION

El modelo contiene 15 entidades y 16 relaciones. La jerarquia principal es:

DEPORTE -> DISCIPLINA -> EVENTO -> EVENTO_EDICION

Las entidades geograficas y de edicion son:

PAIS <- NOC, CIUDAD -> SEDE -> EDICION

La participacion se modela con el grano de una entrada al evento:

EVENTO_EDICION -> PARTICIPACION <- NOC

La relacion muchos-a-muchos entre atletas y participaciones se resuelve con PARTICIPACION_ATLETA. La procedencia de cada registro se conserva mediante ATLETA_FUENTE, relacionada con ATLETA y FUENTE. MEDALLA es un catalogo referenciado opcionalmente por PARTICIPACION.

3.1 Entidades y responsabilidad

Entidad	Responsabilidad
pais	Pais o region normalizada
noc	Comite olimpico historico, incluido el codigo del periodo
ciudad	Ciudad sede
edicion	Edicion de los Juegos, ano y temporada
sede	Relacion entre una edicion y su ciudad
deporte	Categoria deportiva de nivel superior
disciplina	Especialidad dentro de un deporte
evento	Prueba competitiva
evento_edicion	Evento ofrecido en una edicion concreta
medalla	Catalogo de oro, plata y bronce
atleta	Persona unificada
participacion	Entrada de un equipo o atleta en un evento
participacion_atleta	Atletas asociados a una participacion
fuente	Fuente original del registro
atleta_fuente	Trazabilidad del atleta hacia una fuente

3.2 Decisiones de diseno

PARTICIPACION no se modela como una fila por atleta. Un equipo de baloncesto tiene una participacion y varias filas en PARTICIPACION_ATLETA; de esta forma no se cuentan varias veces la misma medalla.

NOC se separa de PAIS porque un pais puede tener varios comites historicos, por ejemplo GER, FRG, GDR y SAA. Esta separacion conserva la historia y permite consultas por pais sin perder el codigo original.

La jerarquia deporte-disciplina-evento permite reconciliar las fuentes que usan catalogos con distintos niveles de detalle. Las tablas de procedencia permiten auditar el proceso de integracion y detectar coincidencias de atletas.

El diagrama editable se encuentra en diagramas/modelo-er-olimpiadas.drawio y su imagen de referencia en diagramas/modelo-er-olimpiadas.png.

4. PLAN DE RESPALDO

4.1 Tecnologia seleccionada

Se utiliza PostgreSQL 17 porque incorpora pg_basebackup --incremental y pg_combinebackup. Los respaldos son fisicos, en formato plano y con WAL transmitido por streaming:

```
text
pg_basebackup -Fp -X stream -c fast
```

La configuracion del contenedor activa:

```
text
summarize_wal=on
wal_level=replica
max_wal_senders=5
```

summarize_wal=on es necesario para que PostgreSQL pueda calcular los bloques modificados desde el manifiesto anterior.

4.2 Cadena de respaldos

Para cada tipo se conserva la siguiente estructura:

```
text
/backups/<tipo>/full
/backups/<tipo>/inc1
/backups/<tipo>/inc2
/backups/<tipo>/inc3
```

inc1 se calcula contra full, inc2 contra inc1 e inc3 contra inc2.
El archivo backup_label y el manifiesto permiten verificar que cada respaldo es realmente incremental.

4.3 Almacenamiento y restauracion

Docker mantiene dos volumenes persistentes e independientes:

- olimpiadas_f2_pgdata: directorio de datos de PostgreSQL;
- olimpiadas_f2_backups: respaldos fisicos.

El procedimiento de desastre elimina solo el volumen de datos. Los respaldos permanecen disponibles para pg_combinebackup, que reconstruye un directorio completo a partir del full y los incrementales requeridos. Luego PostgreSQL realiza la recuperacion usando el WAL incluido.

La separacion de volumenes es una medida de seguridad operacional: el evento que destruye la instancia no debe destruir tambien la copia de respaldo.

5. ESPECIFICACIONES DEL SERVIDOR Y REPRODUCIBILIDAD

La infraestructura requerida es:

Componente	Requisito o configuracion
Sistema operativo	Linux o macOS con Bash
Contenedores	Docker Engine y Docker Compose v2
Base de datos	PostgreSQL 17 Alpine
Python	Python 3 para generar cargas
Puerto publicado	5435
Base y usuario	olimpiadas / olimpiadas_app
Almacenamiento	Aproximadamente 1 GB libre
Volumenes	Datos y respaldos, separados

Las especificaciones variables de la maquina (CPU, memoria, version exacta de Docker y hora del sistema) deben registrarse en la maquina que toma las capturas, con:

```
bash
lscpu
free -h
docker version
date
```

No se sustituyen esos valores por estimaciones: las duraciones dependen del hardware, del sistema de archivos y de la carga concurrente.

6. RESULTADOS TECNICOS DE REFERENCIA

Los resultados versionados del ciclo anio muestran que la restauracion conserva los conteos de las cargas:

Etapa	participacion	atleta	evento_edicion
Carga 01 / full	9,579	11,095	351
Carga 02 / full + inc1	18,597	19,139	698
Carga 03 / full + inc1 + inc2	27,986	26,555	1,015
Carga 04 / full + inc1 + inc2 + inc3	36,901	33,100	1,329

En la medicion de referencia:

Respaldo	Duracion	Tamano
Full	2.31 s	59,268 KB
Incremental 1	2.19 s	38,940 KB
Incremental 2	2.22 s	38,716 KB
Incremental 3	2.29 s	42,940 KB

Restauracion	Archivos	Recuperacion	Total
Full	0.80 s	5.85 s	6.65 s
Full + inc1	2.53 s	5.83 s	8.36 s
Full + inc1 + inc2	2.51 s	5.82 s	8.33 s
Full + inc1 + inc2 + inc3	2.59 s	5.82 s	8.41 s

Los incrementales pesan menos que el full, pero la recuperacion de PostgreSQL permanece aproximadamente constante. La cadena incremental agrega alrededor de 1.7 segundos al tiempo total de restauracion en este conjunto pequeno, debido principalmente a la reconstruccion de archivos con pg_combinebackup.

Estos resultados no deben generalizarse a bases grandes: el tamano reducido hace que el tiempo fijo de arranque y recuperacion domine la medicion. La ventaja principal del incremental se espera en el volumen de datos que debe copiarse despues de la carga inicial.

7. VALIDACION, RIESGOS Y CONCLUSIONES

La validacion combina cuatro controles:

1. conteos de las 15 tablas en cada etapa;
2. muestras de las filas cargadas;
3. integridad referencial mediante consultas de llaves sin padre;
4. comparacion de los conteos originales contra cada restauracion.

Las cargas masivas no ejecutan UPDATE ni DELETE, por lo que se esperan cero tuplas muertas. Los indices pueden mostrar fragmentacion de hojas por splits durante inserciones consecutivas; por eso se mide con pgstatindex y no solo con el tamano de la tabla.

La estrategia recomendada es full mas incrementales encadenados cuando se prioriza reducir el volumen de cada respaldo posterior. Debe aceptarse el costo de reconstruir toda la cadena y conservar todos sus eslabones. Para una recuperacion mas simple y frecuente, un nuevo full puede reducir la dependencia entre respaldos.

La solucion cumple el objetivo de la fase: usa respaldos fisicos reales, almacenamiento persistente separado, restauracion reproducible y validacion cuantitativa de los datos recuperados.

8. REFERENCIAS DEL REPOSITORIO

- README.md: preparacion y comandos generales.
- docs/manual-usuario.md: procedimiento operativo y solucion de problemas.
- docs/modelo-de-datos.md: diccionario, cardinalidades y normalizacion.
- docs/HANDOFF-decisiones.md: decisiones de diseno.
- scripts/03-backup/: respaldos full e incrementales.
- scripts/04-restore/: eliminacion y restauracion.

- scripts/05-validacion/: validacion y fragmentacion.
- logs/: resultados de ejecuciones y tiempos.